

Architecture Document

FinLit India — Gamified App for Financial Knowledge
Programming Project — THWS

Development Team

June 2026

Contents

1	System Overview	2
1.1	Data Flow	2
2	Frontend Architecture	2
2.1	Technology Stack	2
2.2	Project Structure	2
2.3	Application Entry Point	3
2.4	Screen Descriptions	3
2.4.1	Registration Screen	3
2.4.2	Roadmap Screen	3
2.4.3	Lesson Screen	3
2.4.4	Budget Screen	3
2.4.5	Profile Screen	3
3	Backend Architecture	4
3.1	Technology Stack	4
3.2	API Endpoints	4
3.3	Quiz Scoring	4
3.4	Streak Logic	4
4	Data Architecture	4
4.1	Firestore Collections	4
5	Dialogue System	5
6	Key Technical Decisions	6
6.1	No State Management Library	6
6.2	Local Dialogue Content	6
6.3	Dual Budget Persistence	6
6.4	Sequential Lesson Locking	6
6.5	Backend Running Locally	6
7	Known Limitations	6

1 System Overview

FinLit India is a gamified financial-literacy mobile application targeting low-income Indian households. The system follows a classic client-server architecture with three layers: a Flutter mobile frontend, a Python FastAPI backend, and a Google Cloud Firestore database.

Layer	Technology	Location
Mobile frontend	Flutter (Dart)	<code>frontend/lib/</code>
REST backend	Python FastAPI	<code>frontend/backend/</code>
Database	Google Cloud Firestore	Cloud (Firebase)
Local content	JSON asset bundle	<code>assets/data/</code>

1.1 Data Flow

```

Flutter App
|
| HTTP/JSON (package:http)
v
FastAPI Server (localhost:8000)
|
| firebase_admin SDK
v
Google Cloud Firestore
|
| serviceAccountKey.json
v
Firebase Project "finlit-india"

```

One exception: lesson dialogue scripts are bundled as a local JSON asset (`assets/data/dialogues.json`) and loaded directly by the Flutter app without a network request.

2 Frontend Architecture

2.1 Technology Stack

Component	Choice
Language	Dart
Framework	Flutter (SDK $\geq$ 3.10.0), Material 3
State management	<code>setState</code> (no external library)
HTTP client	<code>http</code> ^1.2.0
Local storage	<code>shared_preferences</code> ^2.3.0
Design language	Duolingo-inspired, primary green #58CC02

2.2 Project Structure

```

frontend/lib/
  main.dart           Entry point, routing decision
  models/
    lesson.dart       Lesson + QuizQuestion data models
    dialogue.dart     Dialogue message models (chat format)
    user.dart         UserProfile model
  screens/
    registration_screen.dart  First-launch phone number input
    roadmap_screen.dart      Main screen, lesson path + navigation

```

<code>lesson_screen.dart</code>	Chat dialogue + quiz
<code>budget_screen.dart</code>	50/30/20 budget management
<code>profile_screen.dart</code>	User stats
<code>services/</code>	
<code>api_service.dart</code>	All HTTP calls to the backend
<code>dialogue_service.dart</code>	Loads local JSON dialogue content

2.3 Application Entry Point

On launch, `main.dart` reads the `is_registered` flag from `SharedPreferences`. If true, the app navigates directly to `RoadmapScreen`; otherwise it shows `RegistrationScreen`. The user identifier (`user_id`) is a Firestore document ID stored locally after registration.

2.4 Screen Descriptions

2.4.1 Registration Screen

Collects a 10-digit Indian phone number with a +91 prefix. On submission, calls `POST /auth/register`, stores the returned `user_id` in `SharedPreferences`, and navigates to the roadmap. No OTP verification is performed in the current build.

2.4.2 Roadmap Screen

The main shell of the authenticated app. Renders a `BottomNavigationBar` with four tabs (Home, Shop, Budget, Profile) in an `IndexedStack` so tab state persists while switching. The Home tab fetches lessons and user progress from the API on every load, sorts lessons by `orderIndex`, and applies sequential locking: a lesson is locked unless all preceding lessons are completed. Lesson circles are drawn in a zigzag pattern using five alternating x-positions; the connecting path between circles is rendered with a `CustomPainter` using cubic Bézier curves.

2.4.3 Lesson Screen

Replaced in Sprint 2 with a chat-style dialogue interface. Messages from characters (Sunita, Meena, Raj) appear one by one with a 600 ms animated delay. The user selects responses on behalf of Priya (the protagonist) via choice buttons. Fact cards appear as highlighted green blocks. After the dialogue, the user starts the quiz.

2.4.4 Budget Screen

Allows the user to enter their monthly income and view a breakdown across five categories: Food, Rent, Education, Transport, and Savings. Allocations are initially suggested using the 50/30/20 rule and can be manually adjusted. Budget data is persisted to both `SharedPreferences` (offline cache) and Firestore via the API.

2.4.5 Profile Screen

Displays the user's phone number, a learner badge, and stats. Currently streak, coins, and gems are hardcoded to zero on this screen (a known issue).

3 Backend Architecture

3.1 Technology Stack

Component	Choice
Language	Python 3
Framework	FastAPI
Server	Uvicorn (ASGI)
Validation	Pydantic (v1-style BaseModel)
Database	Firebase Admin SDK + google-cloud-firestore
CORS	allow_origins=["*"]

3.2 API Endpoints

All endpoints are defined in `backend/main.py`.

Method	Path	Description
POST	<code>/auth/register</code>	Register user by phone number
POST	<code>/auth/profile</code>	Save name and language
GET	<code>/lessons</code>	Return all lessons
GET	<code>/lessons/{id}</code>	Return single lesson
POST	<code>/progress</code>	Mark lesson as completed
GET	<code>/progress/{userId}</code>	Get user progress
GET	<code>/quiz/{lessonId}</code>	Get quiz questions for a lesson
POST	<code>/quiz/result</code>	Submit answers, compute score
GET	<code>/user/{userId}</code>	Get user stats (streak, coins, gems)
GET	<code>/budget/{userId}</code>	Get saved budget
POST	<code>/budget</code>	Save budget

3.3 Quiz Scoring

Coins are awarded proportionally: `coins_earned = int(coinsReward × score / total_questions)`. A learner answering half the questions correctly earns half the lesson's coin reward. There is no minimum threshold.

3.4 Streak Logic

Streak is incremented once per calendar day on the first activity (lesson completion or quiz submission). A second activity on the same day does not increment the streak. If no activity occurs for two or more days, the streak resets to 1 on the next activity.

4 Data Architecture

4.1 Firestore Collections

Collection: `users`

Each document represents one registered user.

Field	Type	Description
phone	string	Phone number (identity)
name	string?	Display name
language	string?	Selected language
streak	integer	Current daily streak
coins	integer	Earned coins
gems	integer	Gems (currently always 0)
lastActivity	timestamp	Last activity date (for streak logic)
created_at	timestamp	Registration timestamp

Sub-collections per user:

- `progress/{lessonId}` — `is_completed`, `completed_at`
- `quiz_results/{auto-id}` — `lesson_id`, `score`, `completed_at`
- `budget/current` — income and per-category amounts

Collection: `lessons`

Field	Type	Description
title	string	Lesson title
description	string	Short preview text
content	string	Full article text (legacy)
moduleNumber	integer	Module grouping
sectionName	string	Section label
order_index	integer	Sort order
coinsReward	integer	Coins awarded on completion

Sub-collection: `questions/{auto-id}` — `question`, `option_1` through `option_4`, `correct_option` (1–4).

5 Dialogue System

Lesson content in Sprint 2 is delivered as a scripted chat dialogue between four recurring characters:

Character	Role
Priya	The player character (learner)
Sunita	Priya's friend, same age — shares practical advice
Meena	Older neighbour — closing wisdom
Raj	Priya's husband — appears in budget examples

Dialogues are stored in `assets/data/dialogues.json` and loaded at runtime via `DialogueService`. Each dialogue item has a `type`: `message`, `choice`, or `fact_card`. Choice items present two options; selecting one sets the active branch and filters subsequent messages. Branches converge at items marked `"branch": "merge"`.

The mapping between a Firestore lesson and its dialogue is made via `order_index`: lesson with `order_index = 1` loads dialogue `id = 1` from the JSON file.

6 Key Technical Decisions

6.1 No State Management Library

The app uses `setState` exclusively. Given the current scope — no shared state between screens, each screen re-fetches its own data independently — this is sufficient and keeps the codebase simple for a three-person team.

6.2 Local Dialogue Content

Dialogue scripts are bundled with the app rather than served from the backend. This allows lesson dialogue to be read without a network connection. Quiz questions, by contrast, are stored in Firestore and served via the API, enabling content updates without an app release.

6.3 Dual Budget Persistence

Budget data is saved to both `SharedPreferences` and Firestore. The app tries the API first on load; if it fails, it falls back to the local cache. This provides basic offline resilience.

6.4 Sequential Lesson Locking

Locking logic runs entirely on the client. A lesson is locked if the immediately preceding lesson (by `orderIndex`) is not marked complete in the progress data returned by the API.

6.5 Backend Running Locally

The backend URL is hardcoded as `http://127.0.0.1:8000`. On an Android emulator, this is overridden to `10.0.2.2:8000` (the emulator's alias for the host machine's localhost). The backend must be started manually before running the app.

7 Known Limitations

1. **No authentication security.** Registration creates an account from any phone number with no verification. The `user_id` stored locally is trusted completely.
2. **Hardcoded backend URL.** Not suitable for deployment on physical devices or multiple machines without configuration.
3. **Service account key in repository.** The Firebase Admin SDK credentials file is committed to version control — a security risk that must be resolved before any production deployment.
4. **Profile screen shows zero stats.** `ProfileScreen` does not call `getUserStats()`; streak, coins, and gems are hardcoded to 0.
5. **Dialogue-lesson mapping is fragile.** The implicit contract between `order_index` and dialogue `id` breaks if Firestore data is reordered.
6. **Gems are never awarded.** The field exists in the data model and UI but no backend logic increments it.
7. **Unused backend modules.** `user.py`, `lesson.py`, `question.py`, `quiz_result.py`, and `user_progress.py` define standalone FastAPI instances but are never imported by `main.py`.